

C Programming / C 程式設計

期末專題報告 Term Project Report

Clawer C Data Normalization Engine

C 語言大學排名資料正規化引擎

學號	姓名
b11402037	高恩在
b11402053	林秉學

2026 年 6 月

1. Introduction | 簡介

1.1 背景與動機

上游爬蟲系統自 QS World University Rankings、Times Higher Education (THE) 與 Shanghai Ranking (ARWU) 等多個大學排名來源抓取資料。由於各來源的欄位慣例不一致，爬蟲輸出的 CSV 格式高度異質，無法直接進行跨來源的比較與統計分析。主要的異質性來源包括：

- **大學名稱**：大小寫不一、知名縮寫 (MIT、ETH、KAUST 等)、標點與多餘空白差異。
- **國家名稱**：縮寫 (U.S.A.、UK、PRC)、官方長名 (Federal Republic of Germany) 與「The X」前綴變體 (The Netherlands)。
- **排名格式**：純數字 (53)、區間 (101-150)、文字前綴 (Top 100、Rank 53、=201、#10、201+) 等多種寫法。
- **分數欄位**：含前後空白、前導小數點 (.5)、空字串與 N/A 等無效值。

1.2 研究問題

在不修改下游分析邏輯的前提下，如何將異質性爬蟲輸出 (多種大學名稱格式、排名區間字串、不一致的國家縮寫) 轉換為可直接比較的數值紀錄？

1.3 設計選擇

本專題以 C 語言 (C11, gcc -Wall -Wextra) 實作核心引擎，而非採用 Python / pandas。主要考量為：大量 CSV 批次處理時可維持低記憶體佔用與可預期的執行時間；且無 Python 執行期依賴，便於與上游爬蟲管線的部署整合。

1.4 系統範圍

- **輸入**：爬蟲輸出 CSV (11 欄原始格式)。
- **輸出**：標準化 CSV (5 欄：University, Country, Rank Min, Rank Max, Overall Score)。
- **邊界**：單機、檔案導向；無資料庫伺服器、無 Web API、無外部網路服務依賴。

2. System Design | 系統設計

2.1 系統架構與資料流

系統採單向資料流的 pipeline 架構：CSV Reader 讀入原始資料並填入記憶體中的 UniversityRecord 陣列，Normalization Pipeline 依序呼叫四個解析模組就地正規化，最後由 CSV Writer 輸出標準化結果。各模組以標頭檔 (include/) 定義公開介面，彼此低耦合：

```
raw_universities.csv
→ csv_reader.c      ( header 驗證 / quoted CSV 解析 / BOM 清理 )
→ UniversityRecord[] ( raw 欄位 )
→ normalizer.c      ( pipeline 協調器 )
    ├── name_normalizer.c    名稱清理與縮寫保留
    ├── country_normalizer.c 國家別名映射
    ├── rank_parser.c        排名區間解析
    └── score_parser.c       分數提取
→ csv_writer.c        ( quoting / escaping )
→ normalized_universities.csv
```

模組	檔案	主要責任
CLI 主程式	src/main.c	互動式選單，協調載入、正規化、預覽、匯出
CSV Reader	src/csv_reader.c	header 驗證、quoted CSV 解析、BOM 清理、不合法列略過
Record 模型	include/record.h	保存 raw 與 normalized 欄位、rank_min/max、score
Pipeline 協調器	src/normalizer.c	逐筆呼叫名稱、國家、排名、分數四個解析模組
Name Normalizer	src/name_normalizer.c	名稱清理、大小寫調整、知名縮寫保留
Country Normalizer	src/country_normalizer.c	國家別名映射與標準化 (38+ 條目)
Rank Parser	src/rank_parser.c	解析 Top 100、Rank 53、101-150、=201、#10、201+
Score Parser	src/score_parser.c	字串分數提取，失敗回傳哨兵值 -1

CSV Writer	src/csv_writer.c	標準化輸出，必要時 quoting / escaping
共用工具	src/utls.c	trim、collapse spaces、title case、縮寫偵測

2.2 CSV Schema 設計

本系統採檔案導向設計，以 CSV 作為持久層，不依賴外部資料庫伺服器，降低部署複雜度並貼合爬蟲批次輸出的使用情境。輸入與輸出格式如下：

```
輸入 ( 11 欄 ) : QS Rank,University,Country,GMAT,GRE,GPA,IELTS,TOEFL,
                Duolingo,Overall Score,URL
輸出 ( 5 欄 ) : University,Country,Rank Min,Rank Max,Overall Score
```

實際資料範例 (正規化前 → 後)：

```
raw | University of California Berkeley (UCB) , usa ,12 , 14 , 91.7
norm | University of California Berkeley (Ucb),United States,12,14,91.70
```

2.3 核心資料結構

UniversityRecord 同時保存 raw 與 normalized 欄位，便於前後對照、預覽與除錯：

```
typedef struct {
    /* Raw input fields */
    char raw_name[150];
    char raw_country[80];
    char raw_rank[50];
    char raw_score[50];
    /* Normalized fields */
    char normalized_name[150];
    char normalized_country[80];
    int rank_min, rank_max;
    double score;
} UniversityRecord;
```

2.4 關鍵設計決策

- **國家別名對照表**：靜態 key → value 查表。先以 normalize_basic (去標點、轉小寫、壓縮空白) 做正規化，再比對別名表；涵蓋縮寫、官方長名與「The X」前綴變體，目前共 38+ 條目。
- **排名區間表示**：以 rank_min / rank_max 兩個整數欄位統一表達單值與區間 (53 → [53,53] ; 101-150 → [101,150] ; Top 100 → [1,100])，下游可直接做數值比較。
- **哨兵值設計**：無法解析的分數標記為 -1，由 writer 層轉為空欄位輸出，避免髒資料混入下游統計。
- **Reader 防衛機制**：header 欄位數與欄名驗證 (不符即拒絕載入)、RFC 4180 quoted CSV 解析、UTF-8 BOM 清理、不合法列略過。

3. Implementation | 實作

3.1 開發環境與建置

開發語言為 C11，編譯器為 gcc (編譯選項 -Wall -Wextra -std=c11)，以 Makefile 管理主程式與九個測試套件的建置。macOS / Linux 環境直接使用 make；Windows 環境 (MSYS2 gcc) 可用以下指令直接編譯：

```
gcc -Wall -Wextra -std=c11 -Iinclude src/main.c src/normalizer.c \
    src/csv_reader.c src/csv_writer.c src/name_normalizer.c \
    src/country_normalizer.c src/rank_parser.c src/score_parser.c \
    src/requirement_parser.c src/utils.c -o build/clawer_normalizer.exe
```

另針對 Windows 主控台預設使用 CP950 (Big5) 編碼導致 UTF-8 中文字串顯示為亂碼的問題，於 main.c 加入條件編譯處理：程式啟動時呼叫 SetConsoleOutputCP(CP_UTF8) 切換主控台編碼，並以 #ifdef _WIN32 包覆，確保 macOS / Linux 編譯不受影響。

3.2 互動式操作流程

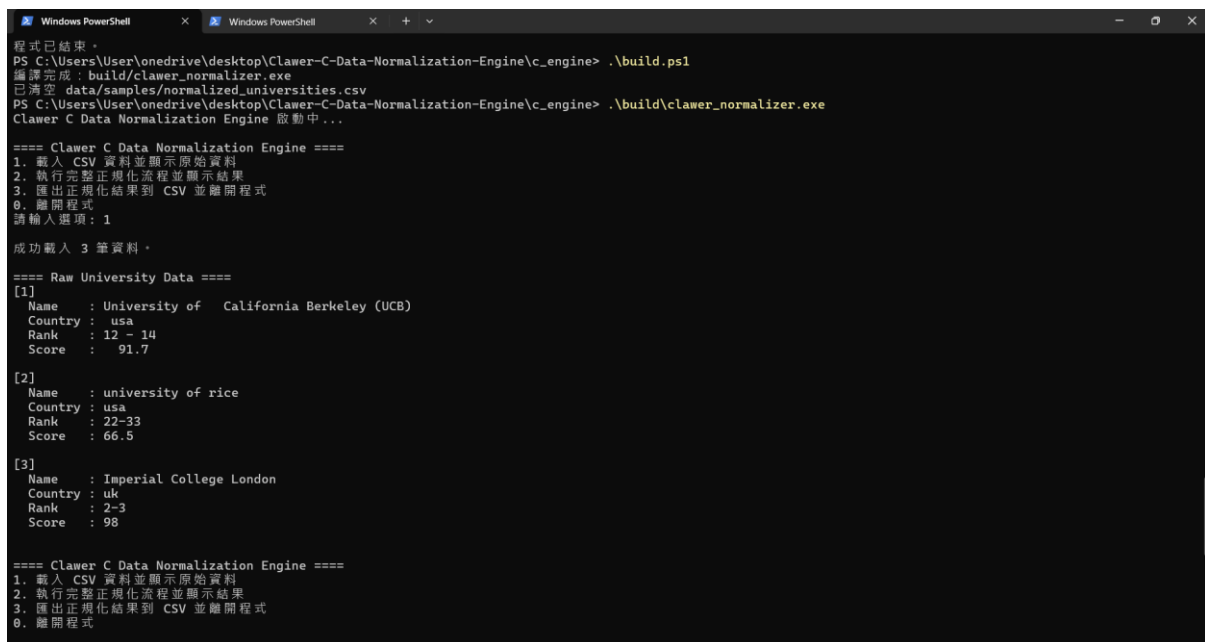
主程式提供互動式 CLI 選單，並具備狀態防呆 (未載入資料或未正規化時，擋下後續操作)：

```
==== Clawer C Data Normalization Engine ====
```

1. 載入 CSV 資料並顯示原始資料
2. 執行完整正規化流程並顯示結果
3. 匯出正規化結果到 CSV 並離開程式
0. 離開程式

選單採同步複合動作設計：載入後立即顯示原始資料、正規化後立即顯示結果、匯出成功後自動離開程式。標準操作序列為 1 → 2 → 3，輸出檔為

data/samples/normalized_universities.csv。以下為 Windows 環境的實際執行畫面：



```
Windows PowerShell
程序已結束。
PS C:\Users\User\onedrive\desktop\Clawer-C-Data-Normalization-Engine\c_engine> .\build.ps1
編譯完成：build\clawer_normalizer.exe
已清空 data/samples/normalized_universities.csv
PS C:\Users\User\onedrive\desktop\Clawer-C-Data-Normalization-Engine\c_engine> .\build\clawer_normalizer.exe
Clawer C Data Normalization Engine 啟動中...

==== Clawer C Data Normalization Engine ====
1. 載入 CSV 資料並顯示原始資料
2. 執行完整正規化流程並顯示結果
3. 匯出正規化結果到 CSV 並離開程式
0. 離開程式
請輸入選項：1
成功載入 3 筆資料。

==== Raw University Data ====
[1]
Name : University of California Berkeley (UCB)
Country : usa
Rank : 12 - 14
Score : 91.7
[2]
Name : university of rice
Country : usa
Rank : 22-33
Score : 66.5
[3]
Name : Imperial College London
Country : uk
Rank : 2-3
Score : 98

==== Clawer C Data Normalization Engine ====
1. 載入 CSV 資料並顯示原始資料
2. 執行完整正規化流程並顯示結果
3. 匯出正規化結果到 CSV 並離開程式
0. 離開程式
```

圖 1：以 build.ps1 編譯（同步清空舊輸出）後執行，選項 1 載入 CSV 並顯示原始資料

```

Windows PowerShell
2. 執行完整正規化流程並顯示結果
3. 匯出正規化結果到 CSV 並離開程式
0. 離開程式
請輸入選項: 2

資料正規化完成。

==== Normalized University Data ====
[1]
Name      : University of California Berkeley (UCB)
Country   : United States
Rank Min  : 12
Rank Max  : 14
Score     : 91.70
[2]
Name      : University of Rice
Country   : United States
Rank Min  : 22
Rank Max  : 33
Score     : 66.50
[3]
Name      : Imperial College London
Country   : United Kingdom
Rank Min  : 2
Rank Max  : 3
Score     : 98.00

==== Clower C Data Normalization Engine ====
1. 載入 CSV 資料並顯示原始資料
2. 執行完整正規化流程並顯示結果
3. 匯出正規化結果到 CSV 並離開程式
0. 離開程式
請輸入選項: 3

已成功輸出到: data/samples/normalized_universities.csv

程式已結束。
PS C:\Users\User\onedrive\desktop\Clower-C-Data-Normalization-Engine\c_engine> |

```

圖 2：選項 2 正規化並顯示結果，選項 3 匯出 CSV 後自動離開程式

3.3 正規化規則與實例

模組	原始輸入 (raw)	正規化輸出 (normalized)
名稱	University of California Berkeley (UCB)	University of California Berkeley (Ucb)
名稱	KAUST · hkust · eth	KAUST · HKUST · ETH (縮寫保留全大寫)
國家	usa · U.S.A. · UK	United States · United Kingdom
國家	The Netherlands · PR China	Netherlands · China (Mainland)
排名	Top 100 · #10 · =201	[1,100] · [10,10] · [201,201]
排名	101-150 · 150-101	[101,150] (反轉區間自動交換)
分數	" 91.7 " · ".5"	91.70 · 0.50 (前導小數點修正)
分數	" " · "N/A" · "abc"	-1 (哨兵值 · 輸出層轉空欄)

規則涵蓋 6 種排名格式與大小寫不敏感比對 ("TOP 100" 等同 "Top 100")，並拒絕負數排名、TOP 0 與整數溢位 (以 strtol 搭配合理上限防護)。

3.4 弱點導向測試方法論

本專題採弱點導向的系統性實驗測試，每輪 Session 依四個步驟進行：（1）靜態分析——通讀模組程式碼並標記可疑邏輯；（2）探針程式——撰寫 probe 重現可疑行為；（3）修正——以最小修改修復根因；（4）回歸驗證——新增 regression 測試套件並全量重跑。累計完成六輪 Session，共 17 項修正（代號 A–T）：

代號	Session	模組	修正說明
A	1/2	country_normalizer	新增 USA/UK/SG 等基本別名
B	1/2	country_normalizer	China/Hong Kong/Taiwan/Korea 別名
C	2	rank_parser	#N 井字號前綴格式支援
D	2	rank_parser	負數排名拒絕
E	3	country_normalizer	含連字號國名（Timor-Leste）保護
F	3	utils	title case 連接詞清單補全
G	3	name_normalizer	移除 Tech→Technology 破壞性展開
H	3	csv_writer	排名空欄輸出逗號數修正
I	4	rank_parser	TOP N / RANK N 大小寫不敏感
J	4	country_normalizer	Korea Rep. 等 19 個別名
K	4	name_normalizer / utils	ETH/UBC/HKUST/TUM 等縮寫擴充
L	4	rank_parser	sscanf 整數溢位 → strtol + 上限
M	5	csv_reader	rank 欄位部分空白智慧組合
P	5	rank_parser	TOP 0 拒絕（ $N \geq 1$ ）
Q	5	rank_parser	反轉區間 B-A 自動交換
R	6	country_normalizer	The X 前綴 + PR China + 38 個官方長名
S	6	score_parser	前導小數點 .5 → 0.5（原誤判 5.0）
T	6	name_normalizer / utils	KAUST / HKU 縮寫擴充

3.5 測試結果

九個測試套件（make test_all）共 602 項測試全數通過：

測試套件	通過 / 總計	驗證重點
test_normalizer	17 / 17	各模組基礎功能與 CSV 讀寫 round-trip
test_extreme	139 / 139	極端與邊界輸入
test_scale	31 / 31	資料規模與長字串
test_weakness	43 / 43	弱點導向探針案例
test_regression2-6	372 / 372	Sessions 2-6 修正之回歸防護
合計	602 / 602	—

4. Conclusion | 結論

4.1 主要成果

- 完成 C11 正規化引擎：CSV Reader → 四大解析模組 → CSV Writer 的完整 pipeline，可將異質爬蟲輸出轉換為可直接比較的標準化紀錄。
- 602 項單元 / 回歸測試全數通過，涵蓋基礎功能、極端值、資料規模與弱點情境。
- 六輪弱點導向修正（A-T 共 17 項）建立了完整的回歸防護網，每項修正均有對應的 regression 測試確保不會再發。
- 支援跨平台建置與執行（macOS / Linux 的 make 流程與 Windows 的 gcc 直接編譯），並解決 Windows 主控台中文編碼問題。

4.2 已知限制

- ASCII-only：名稱與國家正規化僅處理 ASCII 字元，É / ä / ã 等非 ASCII 字元直接傳遞，不做 Unicode 正規化（NFC/NFD）。
- requirement_parser 尚未實作，GMAT / GRE / GPA / IELTS / TOEFL / Duolingo 等入學要求欄位目前不做解析。
- %.2f 捨入行為：score = 99.999 會輸出為 100.00（四捨五入超出滿分），屬 C 標準庫正常行為，暫以格式一致性為優先。

4.3 未來工作

- 引入 utf8proc / ICU 等 Unicode 函式庫，處理非 ASCII 字元正規化。
- 國家別名表外部化：改為自外部 mapping 檔載入，取代硬編碼，建立可維護的更新流程。
- 與上游爬蟲系統整合並導入 CI 自動化流程（爬蟲 → 正規化引擎 → 下游資料庫），同時提供 library 形式介面（libnormalizer）。
- 效能基準測試（吞吐量、記憶體佔用），對照 Python/pandas 實作驗證 C 設計選擇的優勢。

References | 參考文獻

1. QS World University Rankings, <https://www.topuniversities.com/>
2. Times Higher Education (THE) World University Rankings, <https://www.timeshighereducation.com/>
3. ShanghaiRanking / Academic Ranking of World Universities (ARWU), <https://www.shanghairanking.com/>
4. ISO 3166-1, Codes for the representation of names of countries and their subdivisions.
5. RFC 4180, Common Format and MIME Type for Comma-Separated Values (CSV) Files.
6. Unicode Standard Annex #15, Unicode Normalization Forms.
7. ISO/IEC 9899:2011, Information technology — Programming languages — C (C11).
8. GCC, the GNU Compiler Collection — Documentation, <https://gcc.gnu.org/onlinedocs/>
9. GNU Make Manual, <https://www.gnu.org/software/make/manual/>
10. 本專案內部文件：README.md、c_engine/docs/architecture.md、test-report.md
(Session 1–6 累計測試報告) 。